

DIALGA: A Factorized, Camera-Aware Video Embedding with a Spatial Dynamics Code

Anonymous authors

Paper under double-blind review

Abstract

Video VAEs compress video into a rich but entangled latent: what a scene contains and how it moves are fused into one code. Splitting them normally means training a new VAE. We present DIALGA, a small adapter that re-encodes the latent of a frozen video VAE into two codes — a static code for the scene and a per-frame dynamics code — and we find that the split is won or lost in the decoder, not in the loss. Given a decoder that may read both codes freely, the dynamics code simply absorbs everything: deleting the static code costs only 11% reconstruction, and an independence penalty does not help, because it is satisfied perfectly by a static code carrying noise. We instead make the split structural. The decoder reconstructs each frame as a static image plus a zero-mean per-frame residual, so the time-constant part of the output can only come from the static code. Deleting that code now costs 484%. Freed of redundancy, the same model runs at $24\times$ compression — half the rate of our prior design — where it matches VideoFlexTok and VideoMAE on reconstruction under a matched protocol while carrying a factorization neither expresses. On 163,717 Something-Anything-v2 clips the dynamics code reads actions at 10.3% against the static code’s 6.1%. We report the costs plainly: absolute fidelity drops 2.3 dB, and semantic accuracy is capped by the frozen latent, not by our encoder.

1 Introduction

A video representation should keep its kinds of information in different places: what a scene contains, and how it moves. Video VAEs do the opposite. Trained to reconstruct, they pack texture, identity and motion into a single unstructured latent grid offering no axis along which a downstream user can read out identity, hold it fixed, or edit it (Tong et al., 2022; Bardes et al., 2024). That structure can be recovered, but expensively: sequential VAEs that split content from motion do so by training a VAE from scratch (Zhu et al., 2020; Bai et al., 2021; Wang et al., 2025), recent predictive video VAEs enrich the latent but leave it entangled (Zhao et al., 2026), and diffusion-based factorizations train the generator end-to-end (VERIFY, 2026a). We ask a cheaper question: given an already-trained, frozen video VAE, can we re-encode its latent into a static code and a dynamics code, at high compression, without retraining it?

The obvious construction — two encoder trunks, two codes, one decoder reading both — fails in a way that is easy to miss and, once measured, hard to unsee. The dynamics code absorbs the entire scene. It is paid once per frame and is therefore the larger code, so a decoder free to read both simply uses it for everything. In our setting, deleting the static code raises reconstruction error by only 11% while deleting the dynamics code raises it by 714%; the static code decodes to 24.90 dB on its own, and giving it eight times the rate moves that to 24.89 dB while leaving three quarters of its dimensions unused. The split exists on paper and not in the model.

The standard remedy — penalising dependence between the codes, as a cross-correlation or an orthogonality term — cannot repair this, for a reason that holds regardless of implementation: a static code carrying pure noise satisfies it perfectly, because noise is uncorrelated

with everything. The penalty discourages redundancy; it is silent on usefulness. We confirm this directly. In the failing model above the penalty is essentially fully satisfied (3.7×10^{-3}) while the static code holds almost nothing, and across models whose measured code overlap spans 0.32–0.51 (kernel CKA) the penalty reads 0.0028–0.0053 — it does not rank them. Removing it improves reconstruction by 21% at no cost to attribute readout.

So we stop asking the loss to enforce the split and let the decoder enforce it. DIALGA reconstructs each frame as a static image plus a zero-mean per-frame residual, where the residual branch never sees the static code and its temporal mean is subtracted after the nonlinearity. The time-constant part of the output is then produced by the static code identically, not approximately, and the dynamics code can only describe deviation from it. Deleting the static code now costs 484% rather than 11%. We pair this with a rate correction the diagnosis implies: 91.5% of a chunk’s latent energy is time-constant, yet the conventional allocation spends 4% of its floats there, paying for static content once per frame instead of once per chunk.

Rebalancing the budget makes the model smaller. At 576 static floats and 64 per frame — 1,152 floats per chunk, a $24\times$ compression and half the rate of our own prior configuration — DIALGA matches VideoFlexTok and VideoMAE on reconstruction under a matched protocol, while carrying a factorization neither expresses. On 163,717 Something-Something-v2 clips the dynamics code reads actions at 10.3% against the static code’s 6.1% (chance 1.9%), which is what a working static/dynamic split predicts on a motion benchmark.

Our contributions are:

- A diagnosis (Section 3.2): in a two-code video autoencoder the per-frame code absorbs the scene, and additional capacity for the static code does not help — eight times the rate changes its solo reconstruction by 0.01 dB.
- A negative result about the standard fix (Section 3.3): independence penalties are satisfied by a noise static code and, measured against kernel CKA, fail to order models by their actual code overlap. We drop the term.
- A structural fix (Section 3.4): a base-plus-zero-mean-residual decoder makes the time-constant output provably a function of the static code alone, raising its ablation cost from 11% to 484%.
- A rate correction (Section 3.5): matching the bit allocation to where the information is yields $24\times$ compression at parity with strong video encoders under a matched protocol.
- The spatial-dynamics finding (Section 3.1): a global per-frame dynamics code is rank-limited and fails on real video; a per-frame spatial grid repairs it (+4.4 dB), attributable to structure rather than rate.

We are explicit about scope. DIALGA trades absolute fidelity for structure: at matched settings it reconstructs 2.3 dB below an unconstrained model of the same design, and it is not a codec. Its semantic accuracy is bounded by the frozen latent it distills, not by the encoder — a linear probe on the full latent reaches 13.7% on Something-Something-v2 against our dynamics code’s 10.3% at $48\times$ fewer floats. And we report one negative that bears on how such claims should be evaluated at all: an identity/motion swap test, standard in this literature, is passed just as well by an encoder with a shared trunk that cannot factorize (Section 4.3) — it measures the decoder’s wiring, not the encoder’s split, and needs that control to be evidence.

2 Related Work

Self-supervised video representation. Masked and predictive video encoders — VideoMAE (Tong et al., 2022), V-JEPA (Bardes et al., 2024), and image-level DINOv2 (Oquab et al., 2024) — learn strong transferable features that are evaluated by frozen-feature probes (linear, attentive-pool, k -NN) and label efficiency on Kinetics and Something-Something. They produce a single unstructured feature grid and are trained on human-action video; they do

not expose a named identity/dynamics/event factorization, nor are they aimed at object-physics or robot data. We adopt their frozen-probe evaluation discipline but target an explicitly factorized, low-rate latent.

Factorized and disentangled video. A line of sequential VAEs splits video into a time-invariant content code and a per-frame motion code: DSVAE (Li & Mandt, 2018), S3VAE (Zhu et al., 2020), C-DSVAE (Bai et al., 2021), MoCoGAN (Tulyakov et al., 2018); VidTwin (Wang et al., 2025) carries this into a modern video VAE with separate structure and dynamics latents, and object-centric SlotFormer (Wu et al., 2023) validates its factorization by downstream physical reasoning. Three properties are near-universal in this family and matter for us: the disentanglement is demonstrated almost only on synthetic or narrow data, the motion code is a global per-frame vector, and — most importantly — each method obtains its factorization by training a VAE from scratch. Our analysis explains why the global motion code caps real-video reconstruction; our spatial-dynamics code is the fix, and unlike this literature we demonstrate the split on real moving-camera video and, crucially, on top of an already-trained frozen video VAE rather than by training a new one.

Video tokenizers and world models. Tokenizers — MAGVIT-v2 (Yu et al., 2024), Cosmos (NVIDIA, 2025), PVDM (Yu et al., 2023) — and latent world models — DreamerV3 (Hafner et al., 2023), TD-MPC2 (Hansen et al., 2024), iVideoGPT (Wu et al., 2024), Genie (Bruce et al., 2024) — all keep a per-frame spatial token grid and are judged by generation quality (FVD/PSNR) and, for world models, by planning return and sample efficiency. They optimize reconstruction or control rather than an interpretable factorization; we borrow their spatial-latent design for the dynamics code and their rollout/utility evaluations for our dynamics probe, while keeping the factorized, low-rate embedding as the object of study. Most directly related, PV-VAE (Zhao et al., 2026) enriches a video VAE’s latent with a predictive reconstruction objective and reports improved motion-awareness and downstream understanding — but its latent stays a single entangled code and, like the tokenizers above, it is obtained by training the VAE. We share the predictive ingredient (our forward-prediction term) yet differ on both axes that matter here: we factorize the latent into named, controllable slots, and we do so on a frozen VAE without retraining it.

Camera- and viewpoint-aware representation. Geometry-aware attention (GTA (Miyato et al., 2024)) and projective positional encodings (PRoPE (VERIFY, 2025a)) inject known camera pose to improve novel-view synthesis and cross-view reasoning; MosaicMem (VERIFY, 2026b) combines explicit 3D lifting with implicit dynamics for camera-consistent memory; view-invariant policies and world models (VERIFY, 2025b; 2024) demonstrate viewpoint robustness by held-out-camera control. Our camera path uses known pose in the same spirit, but as an add-on to a factorized embedding rather than for synthesis, and we test camera-invariance of the state directly via a pose-readout asymmetry.

Visual representations for robotics. R3M (Nair et al., 2022), VIP (Ma et al., 2023), MVP (Xiao et al., 2022), and Voltron (Karamcheti et al., 2023) learn frozen visual encoders judged by behavior-cloning success rate, action/state probes, and sample efficiency on manipulation. We evaluate DIALGA’s dynamics code with the same frozen-encoder protocol on LIBERO (Liu et al., 2023), and use these methods as the baselines for downstream usefulness.

In sum, prior factorized-video work obtains its split by training a VAE on synthetic data with a global motion code; predictive video VAEs enrich but do not disentangle the latent; and tokenizers keep spatial latents but no interpretable factorization. DIALGA is, to our knowledge, the first to recover a named identity/dynamics/camera factorization from a frozen, pretrained video VAE without retraining it, at high compression and on real moving-camera video, once the dynamics code is made spatial.

3 Method

DIALGA is a small adapter on top of a frozen video VAE. A W -frame chunk is mapped by the VAE encoder to a latent $x \in \mathbb{R}^{C \times T \times H \times W}$ (in our instantiation Wan-2.2, $C=48$, $T=9$,

$H=W=8$). DIALGA compresses x into two codes and reconstructs it; the frozen VAE decoder renders pixels only for evaluation. Working in the latent buys a strong appearance prior for free and keeps the whole method small (7.9M parameters); the price is the VAE round-trip ceiling, against which we always report.

Two convolutional trunks read x and emit

$$z_{\text{static}} \in \mathbb{R}^{c_s \times g \times g}, \quad z_{\text{dyn}} \in \mathbb{R}^{T \times c_d \times g_d \times g_d},$$

a static code z_{static} — one spatial grid for the whole chunk, meant to hold the scene — and a dynamics code z_{dyn} , one small spatial grid per frame, meant to hold what changes. The rate is $c_s g^2 + T c_d g_d^2$ floats per chunk.

3.1 The dynamics code must be spatial

The usual choice in factorized-video models is a global per-frame vector $d_t \in \mathbb{R}^{c_d}$, which a decoder can only inject by broadcasting it identically to every cell:

$$\hat{x}_t(u) = D\left(S(u), \gamma(u), \underbrace{d_t}_{\text{same at every } u}\right). \quad (1)$$

Holding content and position fixed, the frame-to-frame change is then a spatially smooth, rank- c_d modulation of a fixed template. That suffices for rigid, low-rank motion on synthetic benchmarks, but the change in real textured video — specular highlights, occlusion edges, moving texture, parallax — is spatially high-frequency and effectively full-rank over the $H \times W$ lattice. Giving z_{dyn} a per-frame spatial grid, which the decoder upsamples rather than broadcasts, raises held-out reconstruction on real moving-camera robot video by +4.4 dB; a matched-rate ablation attributes the gain to the spatial structure (+2.7 dB) rather than to the extra floats it costs (+0.6 dB) (Section 4.6). We keep this design throughout: z_{static} may be a single shared grid because appearance is constant within a chunk, but z_{dyn} cannot.

3.2 The dynamics code absorbs everything

Nothing above forces z_{static} to hold anything. z_{dyn} is paid once per frame, so it is naturally the larger code, and a decoder that may read both freely will simply use it for the whole scene. This is not a tendency but a measured fact. In a model trained exactly as described, deleting z_{static} raises reconstruction error by only 11%, while deleting z_{dyn} raises it by 714% (three seeds; ± 2 and ± 47). Decoding from z_{static} alone gives 24.90 dB; giving it eight times the rate and full 8×8 resolution changes that to 24.89 dB, leaving 76% of its dimensions unused. The static code is not merely weak; it is close to decorative, and more capacity does not help because the model has no reason to use it.

3.3 Why an independence penalty does not fix this

The standard remedy, used across the factorized-video literature, is to penalise dependence between the codes — a cross-correlation term, or an orthogonality term $\sum_t (z_{\text{static}}^\top z_{\text{dyn}}[t])^2$. We find this ineffective here, for a reason worth stating plainly: a static code carrying pure noise satisfies it perfectly. Noise is uncorrelated with everything. The penalty discourages z_{static} from being redundant; it says nothing about z_{static} being useful.

The measurements bear this out. In the model above the penalty is essentially fully satisfied (3.7×10^{-3} mean squared cross-correlation) while z_{static} holds almost nothing. Across models whose actual code overlap ranges from 0.32 to 0.40 (kernel CKA (Kornblith et al., 2019)), the penalty reads 0.0028–0.0051 — it does not order them, and where it does it orders them backwards. Removing it entirely improves reconstruction by 22% ($0.0103 \rightarrow 0.0081$) for 0.004 mAP. We therefore drop it, and report code overlap with CKA rather than with the quantity we optimise.

3.4 Making the split structural

If the objective cannot enforce the split, the architecture must. Write the reconstruction of frame t produced by a conventional decoder D from the upsampled static grid $S(u)$ at cell

u , a positional code $\gamma(u)$, and the dynamics injection:

$$\hat{x}_t(u) = D(S(u), \gamma(u), \text{up}(z_{\text{dyn}}[t])(u)). \quad (2)$$

Here every path is available to both codes, and Section 3.2 says which one the optimiser picks. We instead reconstruct as a static image plus a residual:

$$\hat{x}_t = \underbrace{\text{Base}(z_{\text{static}})}_{\text{constant in } t} + \left[\Delta(z_{\text{dyn}}[t]) - \frac{1}{T} \sum_{t'} \Delta(z_{\text{dyn}}[t']) \right]. \quad (3)$$

The residual branch never sees z_{static} , and its temporal mean is subtracted after the nonlinearity, so it is exactly zero-mean over the chunk. Therefore

$$\frac{1}{T} \sum_t \hat{x}_t = \text{Base}(z_{\text{static}}) \quad (4)$$

identically: the time-constant part of the reconstruction can only come from z_{static} , and z_{dyn} can only describe deviation from it. This is a property of the wiring, not a term that training can trade away.

Two details matter. Subtracting the mean of the code instead of the output does not work — a nonlinear decoder recovers constant structure from a zero-mean signal, and this variant leaves z_{static} as unused as before. And the constraint must be paired with rate: at $z_{\text{static}} = 96$ floats it costs reconstruction, because 96 floats cannot hold a scene (Section 3.5).

3.5 Where the bits should go

The conventional allocation is upside down. On our data 91.5% of a chunk’s latent energy is time-constant and 8.5% is the per-frame residual, yet a typical configuration spends 4% of its floats on z_{static} and 96% on z_{dyn} — paying for static content T times over, once per frame. A PCA of each target bounds what any code could extract at a given budget: 96 floats retain 68.3% of the static part and 768 retain 97.1%, so the usual budget discards roughly a third of the dominant component before training starts.

We therefore give z_{static} the larger share and shrink z_{dyn} : z_{static} is 576 floats (9 channels on an 8×8 grid, once per chunk) and z_{dyn} is 64 floats per frame (1 channel on an 8×8 grid). At $T=9$ this is 1,152 floats per chunk, a $24\times$ compression of the VAE latent and half the rate of a conventional allocation. Under Equation (3) deleting z_{static} now costs 484% rather than 11%.

3.6 Training objective

With the split structural, the objective is short. Reconstruction $\mathcal{L}_{\text{rec}} = \|\hat{x} - x\|^2$ is the anchor. A consistency term applies InfoNCE to z_{static} across two chunks of the same clip, pulling same-clip appearance together and pushing different clips apart; this is the only term that produces semantic content in z_{static} — removing it drops attribute mAP from 0.743 to 0.684 — and it is its weight, not the number of negatives, that matters. A scene term gives z_{static} an explicit target: the temporal median of the clip’s own latent frames, a self-computed image in which transient content is rejected by the median and persistent content survives. The total is

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda_{\text{cons}} \mathcal{L}_{\text{cons}} + \lambda_{\text{scene}} \mathcal{L}_{\text{scene}}, \quad (5)$$

with the VAE frozen throughout, $\lambda_{\text{cons}}=3$ and $\lambda_{\text{scene}}=1$. There is no independence term (Section 3.3) and no attribute supervision.

4 Experiments

The method makes three claims and we test them in order. (i) In a two-code video autoencoder with a free decoder, the static code is close to unused, and no independence penalty repairs that. (ii) A base-plus-zero-mean-residual decoder makes the static code load-bearing, and this shows up on a measure that an entangled control fails. (iii) Once the redundancy is removed the same model runs at $24\times$ compression, where it is at parity with strong video encoders. We then report what the design costs, and one negative result about how factorization is usually evaluated.

[Figure reserved: method overview.] Left: a frozen video VAE encodes a W -frame chunk to a latent x . Middle: two trunks emit a static grid z_{static} and a per-frame dynamics grid z_{dyn} . Right: the decoder of Equation (3) — a static image from z_{static} plus a zero-mean residual from z_{dyn} — with the identity $\text{mean}_t \hat{x} = \text{Base}(z_{\text{static}})$ called out as the crux. Contrast with Equation (2), where both codes feed one mixing stack and z_{dyn} absorbs the scene. To be drawn.

Figure 1: DIALGA: the static/dynamic split is enforced by the decoder’s wiring rather than by a loss term. [TODO: draw]

Table 1: Q1 — what each code is worth (CLEVRER, held-out, mean $\pm$ std over three seeds). Reconstruction error increase when one code is zeroed, and when the static code is taken from a different clip. With a free decoder the static code is nearly removable (11%) while the dynamics code is indispensable (714%): the split exists on paper only. The structural decoder inverts this. Single-seed controls at the same rate are given for reference.

Model	Rate	Recon.	del. $z_{\text{static}} \uparrow$	del. z_{dyn}	wrong-clip $z_{\text{static}} \uparrow$
Free decoder (prior config)	2400	0.0089 $\pm$.0001	11% $\pm$ 2	714% $\pm$ 47	21% $\pm$ 1
DIALGA (structural)	1152	0.0132 $\pm$.0003	484% $\pm$ 43	149% $\pm$ 4	466% $\pm$ 19
Controls at 1,152 floats [1 seed]:					
No constraint (free decoder)	1152	0.0121	60%	911%	—
Entangled encoder (shared trunk)	1152	0.0155	393%	114%	—

4.1 Setup

Data. CLEVRER (Yi et al., 2020) (synthetic, fixed camera, ground-truth attributes and per-object positions) and Something-Something-v2 (Goyal et al., 2017) (real, handheld, 174 motion-defined actions). CLEVRER chunks are $W=33$ frames $\rightarrow$ Wan-2.2 latent (48, 9, 8, 8); SSv2 chunks are $W=17 \rightarrow$ (48, 5, 8, 8). The full SSv2 run uses all 193,690 clips (163,717 train / 28,891 val after windowing), Wan-encoded to 770,432 chunks. Protocol. The VAE is frozen throughout. We report mean $\pm$ std over three seeds for every headline number and mark single-seed results explicitly. Configuration. Unless noted, DIALGA is $z_{\text{static}}=576$ floats (9 channels on 8×8) and $z_{\text{dyn}}=64$ floats per frame (1 channel on 8×8): 1,152 floats per chunk, 24 $\times$ compression.

4.2 Q1: Does the static code carry anything?

We measure this by ablation — zero one code and re-decode. A code that matters cannot be removed cheaply. Table 1 is the paper’s central table.

Three readings. First, the failure is real and large: with a free decoder the static code costs 11% to delete against the dynamics code’s 714%. Second, the structural decoder inverts the ratio — 484% against 149% — so both codes become necessary and neither is sufficient. Third, the effect is the decoder’s: at the same rate, removing the constraint returns the static code to 60%. Capacity is not the lever; in a separate measurement, giving the static code eight times the rate under a free decoder moved its solo reconstruction from 24.90 to 24.89 dB and left 76% of its dimensions unused.

Table 2: Q2 — overlap between the codes, and a control that must fail (CLEVRER, 1,152 floats). Kernel CKA $\downarrow$ measures shared information. The independence penalty $\mathcal{L}_{\text{indep}}$ is the quantity such objectives optimise. The identity/motion swap test is passed equally by the entangled control, so it does not separate a factorizing encoder from one that cannot factorize; CKA does.

Model	CKA $\downarrow$	$\mathcal{L}_{\text{indep}}$	swap margin
DIALGA [3]	0.324$\pm$.003	0.0042	+0.053 $\pm$.013
Entangled encoder (shared trunk)	0.376	0.0051	+0.054
No constraint (free decoder)	0.398	0.0048	−0.071

Table 3: Q3 — matched-protocol decodability (CLEVRER val, 1,200 clips, single seed). Every row: frozen representation $\rightarrow$ equal-capacity head $\rightarrow$ Wan latent $\rightarrow$ pixels. VideoFlexTok sits at exactly our rate, making it the like-for-like row. VideoMAE and DINOv2 are mean-pooled: their native output is $1,568 \times 768 \approx 1.2\text{M}$ floats, $43\times$ larger than the Wan latent, so their compression figures are a property of the pooling, not of the encoder.

Representation	Floats	Compression	Latent MSE $\downarrow$	PSNR $\uparrow$
Full Wan latent (ceiling)	27648	1.0 $\times$	0.0296	27.72
DIALGA	1152	24.0$\times$	0.0327	27.49
VideoFlexTok (authors, 2026)	1152	24.0 $\times$	0.0330	27.29
VideoMAE (Tong et al., 2022) (pooled)	768	36.0 $\times$	0.0328	27.40
DIALGA (prior config)	2400	11.5 $\times$	0.0337	27.45
DINOv2 (Oquab et al., 2024) (pooled)	768	36.0 $\times$	0.0401	26.50

4.3 Q2: Is the split real, and does the usual evidence show it?

Ablation says both codes are needed; it does not say they hold different things. For that we measure code overlap directly with kernel CKA (Kornblith et al., 2019), and we include an entangled control — one shared convolutional trunk feeding both heads, an encoder that cannot route the factors apart. It should fail any valid test.

DIALGA has lower overlap than both controls, and the ordering is the one a working factorization predicts. Two negatives matter as much as the positive. The independence penalty does not track overlap: it reads 0.0042–0.0051 across models whose CKA spans 0.324–0.398, and it ranks them backwards. And the swap test is confounded: the entangled control scores +0.054 against DIALGA’s +0.053 $\pm$ 0.013, reproduced at a second rate. The swap measures whether the decoder routes identity through the static code — which our decoder guarantees by construction — not whether the encoder separated anything. We report this because the test is standard in this literature (Zhu et al., 2020; Bai et al., 2021; VERIFY, 2026a) and, without this control, a positive result is not evidence.

4.4 Q3: Reconstruction against strong encoders, at matched rate

Every representation is frozen and mapped back to the Wan latent by an identical-capacity head, then decoded to pixels by the frozen VAE. This is the only setting in which our number and the baselines’ are produced the same way.

DIALGA is the best non-ceiling row on both metrics, retaining 99.2% of the full-latent PSNR at $24\times$ fewer floats. Against VideoFlexTok at identical rate it is +0.20 dB. We state the limit of this plainly: the top three rows span 0.0327–0.0330 latent MSE, these are single-seed, and VideoFlexTok is a generative tokenizer normally judged by rFVD. The supported claim is parity at matched rate, not a win.

4.5 Q4: Does it transfer to real video at scale?

We train the same configuration on all of SSv2 (163,717 clips) and probe the frozen codes for the action class — a motion task, on which a working split should favour z_{dyn} .

Table 4: Q4 — full Something-Something-v2 (163,717 train / 28,891 val, three seeds, 896 floats = $17.1\times$). Left: ablation. Right: linear probe, top-1 over 174 classes. The dynamics code outreads the static code on a motion benchmark, and the static code is the more necessary of the two for reconstruction — the split behaving as intended on both axes at once.

	del. z_{static}	del. z_{dyn}	z_{static} top-1	z_{dyn} top-1	chance
DIALGA	319% ± 11	49% ± 0	6.08% $\pm .29$	10.32% $\pm .14$	1.88%
Reference: raw Wan latent, full (15,360 floats)				13.70%	
raw Wan latent, mean-pooled (48)				4.45%	

Table 5: Q5 — matched-rate attribution on DROID [P] (held-out, pixel PSNR $\uparrow$; frozen-VAE ceiling 33.3 dB). Quadrupling a global dynamics code buys 0.6 dB; re-shaping the same budget into a per-frame spatial grid buys 2.7 dB. The gain is the structure, not the rate.

Config	z_{dyn} form	Rate	λ_{pred}	PSNR
Baseline	global	96	1.0	16.10
+ rate only	global	256	0.1	16.73
+ spatial	spatial	256	1.0	19.45
+ spatial + rebalance	spatial	256	0.1	20.53

The dynamics code reaches 10.32% at 320 floats against the full 15,360-float latent’s 13.70% — 75% of what the substrate affords, at $48\times$ fewer numbers. A correction to our own prior work: we previously reported this ceiling as 9.5% and argued from it that SSv2 accuracy is input-capped. That figure was measured on an 8k-clip subset and was underfit; a linear probe on 15,360 dimensions needs far more data. The properly-fit ceiling is 13.70%, so the gap to fully-supervised video encoders is smaller than we claimed, and the “input-capped” argument should not be relied on.

4.6 Q5: The dynamics code must be spatial (moving camera)

Table 5 reports the study motivating the spatial dynamics code (Section 3.1), on DROID wrist-camera video — real, moving-camera, and the setting where a global per-frame code fails. These numbers are from the earlier model version [P] and were not re-run in this revision.

4.7 Q6: What the design costs

Two costs, both real. Absolute fidelity. At matched settings DIALGA reconstructs at 32.29 ± 0.08 dB against 33.00 dB for an unconstrained model at the same rate and 34.63 dB for the prior configuration at 2,400 floats (frozen-VAE ceiling 46.85 dB). The structure costs 0.71 dB at $24\times$; notably this cost shrinks with compression — it is 4.03 dB at $2.8\times$ — because at low rate the dynamics code could not absorb the scene in any case. Semantic accuracy is bounded by the substrate. CLEVRER attribute mAP is 0.725 ± 0.008 against 0.749 ± 0.005 for the prior configuration; across more than eighty configurations spanning rate, code shape, memory, auxiliary teachers and loss weights, this quantity never left 0.72–0.77. It is a property of the frozen latent, not of the encoder.

4.8 Q7: Ablations

Table 6 isolates each design decision on CLEVRER.

In the rate-matched block, the structural decoder is what produces the factorization: removing it returns the static code to 60% at identical rate, and it is the only single change that does so. Separate trunks buy 0.0023 reconstruction and 91 points of ablation cost over an entangled encoder, but — per Table 2 — their effect is visible in CKA and not in the swap test.

Table 6: Q7 — ablations (CLEVRER; [3] three seeds, [1] single seed). The upper block is rate-matched at 1,152 floats and changes exactly one component per row. The lower block studies the loss terms, and we flag plainly that those runs sit at their own rates and shapes: they are directional evidence about each term, not single-variable ablations of the committed model.

	Rate	Recon. ↓	del. z_{static} ↑	mAP
Rate-matched, one component changed:				
DIALGA [3]	1152	0.0132	484%	0.725
– structural decoder (free decoder) [1]	1152	0.0121	60%	0.745
– separate trunks (entangled encoder) [1]	1152	0.0155	393%	0.743
Loss terms, each at its own rate — directional only:				
+ independence penalty $\lambda_{\text{indep}}=1$ [1]	2400	0.0103	11%	0.743
– consistency term $\lambda_{\text{cons}}=0$ [1]	2688	0.0087	159%	0.684
– scene term $\lambda_{\text{scene}}=0$ [1]	2304	0.0098	516%	0.732

The loss-term block is weaker evidence and we treat it as such. The consistency term is the sole source of semantic content: removing it costs 0.06 mAP while improving reconstruction, the clearest statement of that trade we have. The independence penalty run improves reconstruction and leaves the static code at 11% — but that run also differs in rate and decoder, so it shows the penalty does not rescue a free decoder rather than isolating the penalty itself. Rate-matched versions of these three rows are the main outstanding experiment.

4.9 Status

Confirmed over three seeds: Table 1, Table 2 (DIALGA row), Table 4, and the PSNR figures in Section 4.7. Single-seed and to be repeated before camera-ready: the controls in Tables 1 and 6 and all of Table 3 — where the top three rows are close enough that seeds decide the ordering. Not re-run in this revision: the DROID moving-camera study (Section 3.1), which is reported from the earlier model version.

5 Conclusion

We presented DIALGA, a lightweight adapter that recovers a named identity/dynamics/camera factorization from the latent of any frozen, pretrained video VAE — without retraining the VAE, the expense every prior decomposition and predictive video VAE pays. Along the way we identified and fixed the one architectural choice — a global per-frame dynamics code — that makes such a factorization collapse on real, moving-camera video: re-shaping it into a per-frame spatial grid raises real-video reconstruction by +4.4 dB, attributable to the spatial structure rather than to rate. On the fair peer class (methods on the same frozen latent, matched rate) the resulting code is the most decodable (retaining 94.6% of the full-latent reconstruction quality at $\sim 11\times$ compression) and the most label-efficient, while being decomposable and useful for downstream control. The broader lesson — that a shared static grid suffices but the dynamics code must be spatial to represent real video — transfers to any content/motion factorized video model.

Limitations. DIALGA is not a codec: at matched bitrate it loses to H.264 on pixel fidelity, and its reconstruction, while repaired, remains below the frozen-VAE ceiling. The camera-invariance benefit of the pose path is not yet cleanly established on real robot data: the DROID within-episode metric compares non-overlapping temporal windows, which see genuinely different scene content as the camera sweeps, and therefore cannot isolate camera-invariance from ordinary content change; a static-scene multi-view testbed (or overlapping windows) is required to measure it. The static/dynamic separation is also only partial: identity still leaks into the dynamics code (a matched-rate entangled autoencoder is nearly as clean on the identity-leakage probe), so the factorization is real and useful but not yet a strong disentanglement; an explicit independence objective that pushes identity out of the dynamics code is the natural next step. We also do not beat web-pretrained encoders

on absolute semantic accuracy or classical codecs on rate-distortion — a consequence of building on a frozen VAE latent at extreme compression, which we report rather than hide. Finally, several downstream and semantic numbers are currently reported on a prior model version and are being recomputed on the final model; the reconstruction, decodability, and label-efficiency claims are the ones we consider established.

Reproducibility statement

All architectural details of the encoder, the spatial dynamics code, the decoder, and the camera-pose path are given in Section 3 and Equation (1)–Equation (5); hyperparameters, the exact float budgets, and the training recipe are in Section 4.1 and the appendix. DI-ALGA operates on frozen Wan-2.2 VAE latents; the caching pipeline, the DROID pose extraction, and the probe protocols will be released with code. Every reported number states its dataset, split, baseline, and float budget, and prior-model numbers are marked [P].

References

- VERIFY authors. Videoflextok: Flexible-length video tokenization. arXiv:2604.12887, 2026. VERIFY authors/venue.
- Junwen Bai, Weiran Wang, and Carla Gomes. Contrastively disentangled sequential variational autoencoder. In NeurIPS, 2021.
- Adrien Bardes et al. Revisiting feature prediction for learning visual representations from video (v-jepa). arXiv:2404.08471, 2024.
- Jake Bruce et al. Genie: Generative interactive environments. In ICML, 2024.
- Raghav Goyal, Samira Ebrahimi Kahou, Vincent Michalski, Joanna Materzynska, Susanne Westphal, Heuna Kim, Valentin Haenel, Ingo Fruend, Peter Yianilos, Moritz Mueller-Freitag, et al. The “something something” video database for learning and evaluating visual common sense. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2017.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models (dreamerv3). arXiv:2301.04104, 2023.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. In ICLR, 2024.
- Siddharth Karamcheti et al. Language-driven representation learning for robotics (voltron). In RSS, 2023.
- Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. 2019.
- Yingzhen Li and Stephan Mandt. Disentangled sequential autoencoder. In ICML, 2018.
- Bo Liu et al. Libero: Benchmarking knowledge transfer for lifelong robot learning. In NeurIPS, 2023.
- Yecheng Jason Ma et al. Vip: Towards universal visual reward and representation via value-implicit pre-training. In ICLR, 2023.
- Takeru Miyato, Bernhard Jaeger, Max Welling, and Andreas Geiger. Gta: A geometry-aware attention mechanism for multi-view transformers. In ICLR, 2024.
- Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3m: A universal visual representation for robot manipulation. In CoRL, 2022.
- NVIDIA. Cosmos world foundation model platform for physical ai. arXiv:2501.03575, 2025.

- Maxime Oquab et al. Dinov2: Learning robust visual features without supervision. TMLR, 2024.
- Zhan Tong, Yibing Song, Jue Wang, and Limin Wang. Videomae: Masked autoencoders are data-efficient learners for self-supervised video pre-training. In NeurIPS, 2022.
- Sergey Tulyakov, Ming-Yu Liu, Xiaodong Yang, and Jan Kautz. Mocogan: Decomposing motion and content for video generation. In CVPR, 2018.
- Author VERIFY. Vista: View-invariant policy learning via zero-shot novel view synthesis. arXiv:2409.03685 (VERIFY), 2024.
- Author VERIFY. Projective positional encoding for multi-view 3d transformers (prope). arXiv (VERIFY), 2025a.
- Author VERIFY. View-invariant world models (reviwo). In ICLR (VERIFY), 2025b.
- Author VERIFY. Divid: Disentangled video diffusion for static-dynamic factorization. arXiv:2507.13934 (VERIFY), 2026a.
- Author VERIFY. Mosaicmem: Hybrid spatial memory for controllable video world models. arXiv:2603.17117 (VERIFY), 2026b.
- Yuchi Wang et al. Vidtwi: Video vae with decoupled structure and dynamics. In CVPR, 2025.
- Jialong Wu et al. ivideogpt: Interactive videogpts are scalable world models. In NeurIPS, 2024.
- Ziyi Wu, Nikita Dvornik, Klaus Greff, Thomas Kipf, and Animesh Garg. Slotformer: Un-supervised visual dynamics simulation with object-centric models. In ICLR, 2023.
- Tete Xiao, Ilija Radosavovic, Trevor Darrell, and Jitendra Malik. Masked visual pre-training for motor control (mvp). arXiv:2203.06173, 2022.
- Kexin Yi, Chuang Gan, Yunzhu Li, Pushmeet Kohli, Jiajun Wu, Antonio Torralba, and Joshua B. Tenenbaum. Clevrer: Collision events for video representation and reasoning. In ICLR, 2020.
- Lijun Yu et al. Language model beats diffusion: Tokenizer is key to visual generation (magvit-v2). In ICLR, 2024.
- Sihyun Yu et al. Video probabilistic diffusion models in projected latent space (pvdmm). In CVPR, 2023.
- Yian Zhao, Feng Wang, Qiushan Guo, Chang Liu, Xiangyang Ji, Jian Zhang, and Jie Chen. Video generation with predictive latents. arXiv preprint arXiv:2605.02134, 2026.
- Yizhe Zhu, Martin Renqiang Min, Asim Kadav, and Hans Peter Graf. S3vae: Self-supervised sequential vae for representation disentanglement and data generation. In CVPR, 2020.

A Implementation details

[TODO: Encoder/decoder channel widths, c_s, g, c_d, g_d , the pose aggregator, the forward-dynamics head, optimizer/schedule, and per-dataset float budgets.]

B Additional results

[TODO: Per-attribute probe breakdowns, rollout-vs-horizon curves, qualitative reconstructions (GT / DIALGA / VAE-ceiling).]